



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Redes neuronales para datos tabulares (actividades)

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

13.1	Introducción	1
13.2	Perceptrones multicapa	2
13.3	Backpropagation	6
13.3.1	Diferenciación hacia adelante y hacia atrás	7
13.3.2	Diferenciación hacia atrás para MLPs	8
13.3.3	Grafos de computación	10
13.4	Entrenamiento de redes neuronales	11
13.4.2	Gradientes que desaparecen y explotan	12
13.4.3	Funciones de activación no saturantes	13
13.4.4	Conexiones residuales	14
13.4.5	Inicialización de parámetros	15
13.4.6	Entrenamiento paralelo	16
13.5	Regularización	17
13.5.1	Terminación temprana	18
13.5.2	Penalización de pesos	18
13.5.3	DNNs dispersas	18

13.5.4 Dropout	22
13.5.5 Redes neuronales Bayesianas	24

13.1. Introducción

- ▶ Un perceptrón multicapa (*multilayer perceptron, MLP*) es sinónimo de red neuronal hacia adelante (*feedforward neural network, FFNN*), pero se considera un caso particular de red neuronal profunda (*deep neural network, DNN*). Indica la respuesta incorrecta:
 1. Una FFNN o MLP es una composición de funciones secuencial (en cadena) que asume datos estructurados o tabulares
 2. Un modelo lineal no se considera una DNN
 3. Una DNN es una composición de funciones diferenciables que transforma una entrada estructurada o no en una salida mediante un grafo dirigido acíclico
 4. Todas son correctas

La 4.

13.2. Perceptrones multicapa

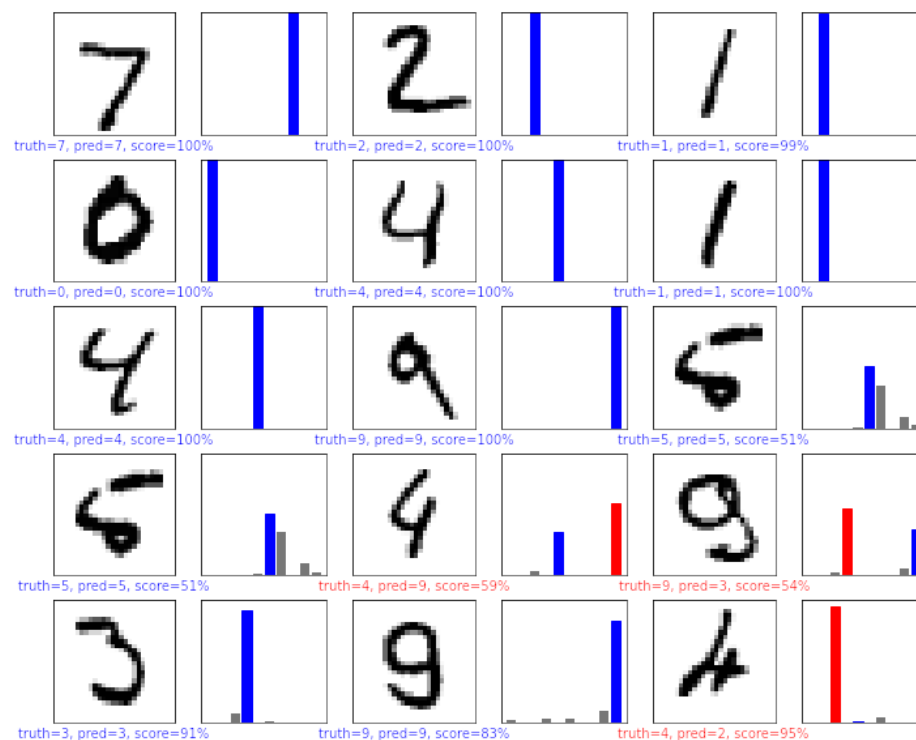
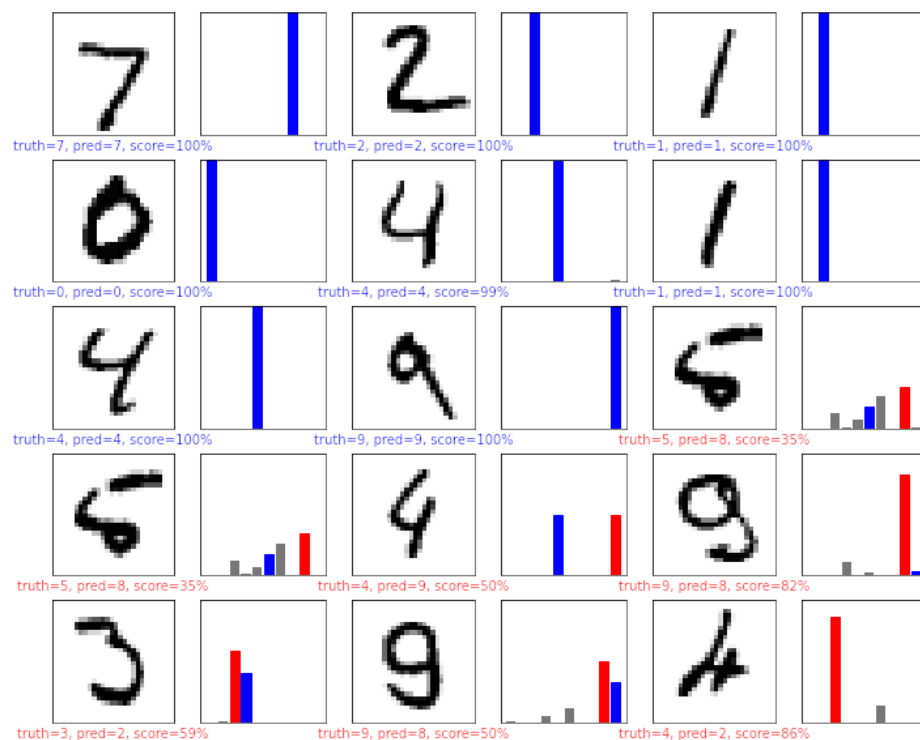
- Reproduce el cuaderno de la [Tabla 13.2 y Figura 13.4](#)

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 128)	16512
dense_2 (Dense)	(None, 10)	1290
Total params: 118,282		
Trainable params: 118,282		
Non-trainable params: 0		

$$100\,840 = (784 + 1) \cdot 128 \qquad 16\,512 = (128 + 1) \cdot 128$$

- ▶ *15 ejemplos de clasificación* (escogidos para incluir errores)
- ▷ Ejemplos tras una (izq.) y dos iteraciones (dcha.) de aprendizaje
- ▷ Azul es correcta; rojo (y gris) incorrecta
- ▷ Tras dos iteraciones, la precisión de test es del 97.1 %



► Reproduce el cuaderno de la [Tabla 13.3](#)

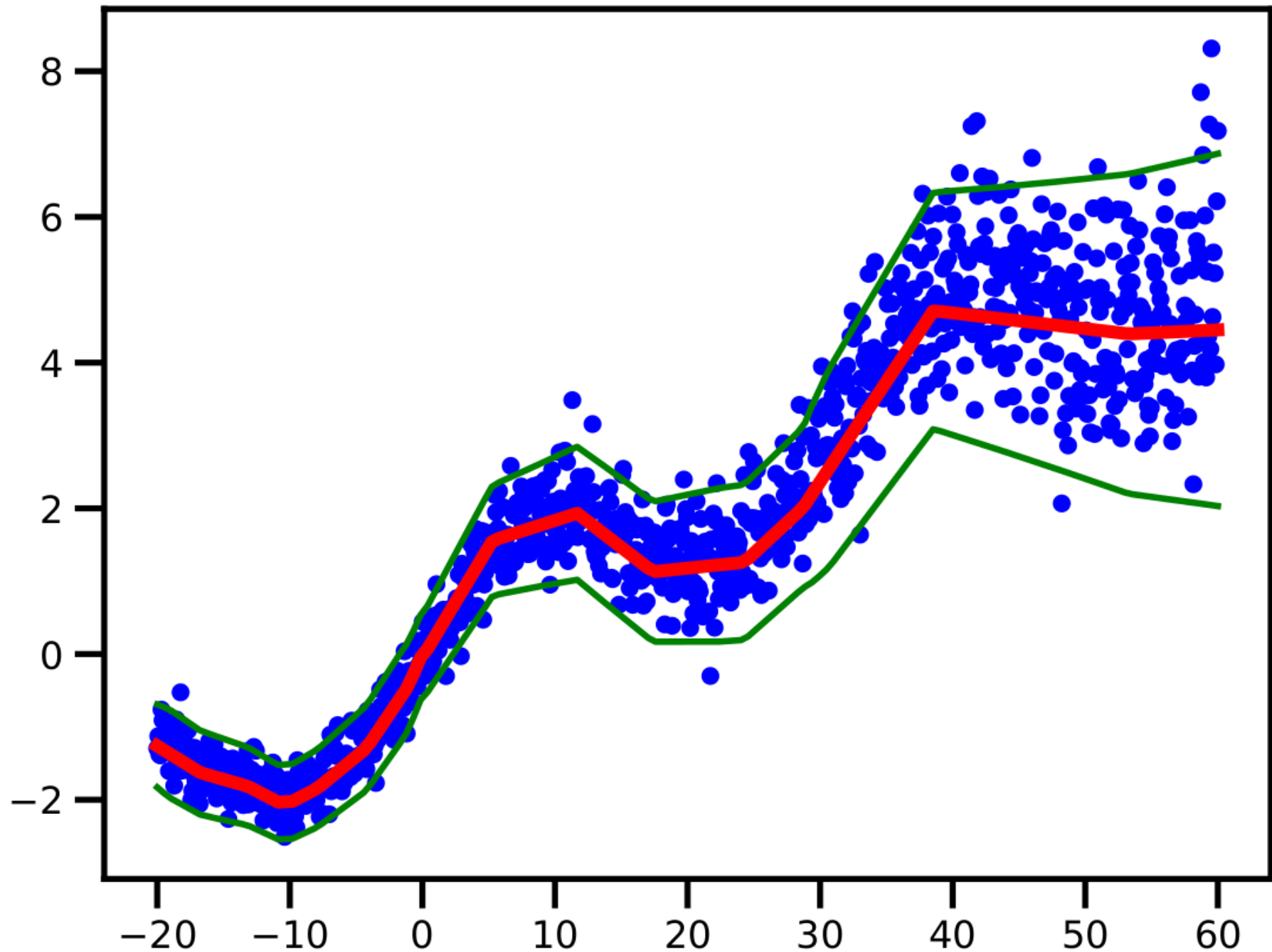
Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, None, 16)	160000
global_average_pooling1d (GlobalAveragePooling1D)	(None, 16)	0
dense (Dense)	(None, 16)	272
dense_1 (Dense)	(None, 1)	17
Total params: 160,289		
Trainable params: 160,289		
Non-trainable params: 0		

$$160\,000 = 10\,000 \cdot 16 \qquad 272 = (16 + 1) \cdot 16$$

Precisión en validación del 86 %

► Reproduce el cuaderno de la [Figura 13.6](#)



13.3. Backpropagation

- ▶ El algoritmo backpropagation (backprop) es una técnica muy popular para entrenar MLPs. Indica la respuesta incorrecta:
 1. Descubierta por Bryson y Ho (1969), re-descubierta por Werbos (1974); y popularizado por Rumelhart, Hinton y Williams (1986)
 2. Calcula el gradiente de una función de pérdida aplicada a la salida de una red con respecto a los parámetros de cada capa; se pasa a un algoritmo de optimización
 3. Puede verse como aplicaciones repetidas de la regla de la cadena si asumimos que el grafo de computación es una cadena lineal simple de capas apiladas
 4. Todas son correctas

La 4.

13.3.1. Diferenciación hacia adelante y hacia atrás

► Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ una composición de K funciones diferenciables, $f_1, \dots, f_K$, cuya Jacobiana en $x \in \mathbb{R}^n$, $\mathbf{J}_f(x)$, queremos hallar mediante diferenciación hacia adelante o hacia atrás. Indica la respuesta correcta:

1. Ambas son igualmente eficientes
2. Si $n > m$, diferenciación hacia adelante es más eficiente
3. Si $n > m$, diferenciación hacia atrás es más eficiente
4. Diferenciación hacia atrás es más eficiente en cualquier caso

La 3.

13.3.2. Diferenciación hacia atrás para MLPs

► Ejercicio 13.1

▷ Sea un MLP con una capa oculta:

$$\mathbf{x} = \text{entrada} \in \mathbb{R}^D \quad (1)$$

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^K \quad (2)$$

$$\mathbf{h} = \text{ReLU}(\mathbf{z}) \in \mathbb{R}^K \quad (3)$$

$$\mathbf{a} = \mathbf{V}\mathbf{h} + \mathbf{b}_2 \in \mathbb{R}^C \quad (4)$$

$$\mathcal{L} = \text{CrossEntropy}(\mathbf{y}, \mathcal{S}(\mathbf{a})) \in \mathbb{R} \quad (5)$$

donde $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{b}_1 \in \mathbb{R}^K$, $\mathbf{W} \in \mathbb{R}^{K \times D}$, $\mathbf{b}_2 \in \mathbb{R}^C$ y $\mathbf{V} \in \mathbb{R}^{C \times K}$, siendo D la talla de la entrada, K el número de unidades ocultas y C el número de clases

▷ Prueba que los gradientes para parámetros y entrada son:

$$\nabla_{\mathbf{V}} \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{V}} \right]_{1,:} = \mathbf{u}_2 \mathbf{h}^t \in \mathbb{R}^{C \times K} \quad (6)$$

$$\nabla_{b_2} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial b_2} \right)^t = \mathbf{u}_2 \in \mathbb{R}^C \quad (7)$$

$$\nabla_{\mathbf{W}} \mathcal{L} = \left[\frac{\partial \mathcal{L}}{\partial \mathbf{W}} \right]_{1,:} = \mathbf{u}_1 \mathbf{x}^t \in \mathbb{R}^{K \times D} \quad (8)$$

$$\nabla_{b_1} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial b_1} \right)^t = \mathbf{u}_1 \in \mathbb{R}^K \quad (9)$$

$$\nabla_x \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{x}} \right)^t = \mathbf{W}^t \mathbf{u}_1 \in \mathbb{R}^D \quad (10)$$

donde

$$\mathbf{u}_2 = \nabla_a \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{a}} \right)^t = (\mathbf{p} - \mathbf{y}) \in \mathbb{R}^C \quad (11)$$

$$\mathbf{u}_1 = \nabla_z \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}} \right)^t = (\mathbf{V}^t \mathbf{u}_2) \odot H(\mathbf{z}) \in \mathbb{R}^K \quad (12)$$

13.3.3. Grafos de computación

- ▶ Una DNN moderna es un grafo de computación, esto es, una composición de funciones diferenciables que transforma una entrada en una salida con un grafo dirigido acíclico. Indica la respuesta incorrecta.
 1. El paso *forward* sigue un orden topológico; de padres a hijos
 2. El paso *backward* sigue un orden topológico inverso, sumando gradientes para caminos múltiples
 3. Los grafos estáticos, calculados en preproceso, han dado paso a los dinámicos, calculados más eficientemente, *just in time*
 4. Todas son correctas

La 4.

13.4. Entrenamiento de redes neuronales

- ▶ El entrenamiento de redes neuronales suele hacerse mediante minimización de la NLL. Indica la respuesta incorrecta.
 1. Es usual añadir un regularizador como la log-prior negativa
 2. Calculamos el gradiente de la pérdida con backprop y se lo pasamos a un optimizador estándar; Adam es muy popular
 3. En la práctica se suelen presentar algunos problemas para los cuales se tienen soluciones más o menos populares
 4. Todas son correctas

La 4.

13.4.2. Gradientes que desaparecen y explotan

- ▶ En el entrenamiento de redes muy profundas, se han propuesto diversas soluciones para que el gradiente no desaparezca o explote. Indica la respuesta incorrecta.
 1. La modificación de las funciones de activación puede evitar que el gradiente sea demasiado pequeño o demasiado grande
 2. El problema de los gradientes que explotan se suele suavizar recortando la magnitud del gradiente hasta un valor fijo dado
 3. El problema de los gradientes que desaparecen se suele suavizar modificando la arquitectura de la red para estandarizar las activaciones de cada capa o hacer que las actualizaciones sean aditivas en lugar de multiplicativas
 4. Todas son correctas

La 4.

13.4.3. Funciones de activación no saturantes

► Indica la respuesta incorrecta.

1. La sigmoide en régimen de saturación favorece gradientes nulos
2. El problema de la “ReLU muerta” se da cuando los logits toman valores negativos grandes, lo que favorece que la ReLU y su gradiente con respecto a los parámetros tiendan a cero
3. La leaky ReLU es una variante no saturante de la ReLU que ataca el problema de la “ReLU muerta”
4. Todas son correctas

La 4.

13.4.4. Conexiones residuales

- ▶ La residual network, ResNet, es un MLP en el que cada capa es un bloque residual de la forma $\mathcal{F}'_l = \mathcal{F}_l(x) + x$. Cada $\mathcal{F}_l$ es una transformación que halla el término residual o delta a añadir a la entrada (perturbada) para generar la salida deseada. Indica la respuesta correcta.

1. $\mathcal{F}_l$ es una transformación lineal superficial
2. $\mathcal{F}_l$ es una transformación lineal profunda
3. $\mathcal{F}_l$ es una transformación no lineal superficial
4. $\mathcal{F}_l$ es una transformación no lineal profunda

La 3.

13.4.5. Inicialización de parámetros

► Dado que el objetivo para el entrenamiento de una DNN no es convexo, la inicialización de parámetros condiciona en gran medida el resultado de su optimización. Indica la respuesta incorrecta.

1. La inicialización Xavier o Glorot toma muestras normales de media nula y varianza $\sigma^2 = 2/(n_{\text{in}} + n_{\text{out}})$, donde n_{in} y n_{out} son el número de conexiones de entrada y salida de una unidad
2. La inicialización LeCun toma $\sigma^2 = 1/n_{\text{in}}$, por lo que equivale a Glorot cuando $n_{\text{in}} = n_{\text{out}}$
3. La inicialización He toma $\sigma^2 = 2/n_{\text{in}}$
4. Todas son correctas

La 4.

13.4.6. Entrenamiento paralelo

- ▶ Actualmente se usan múltiples máquinas (GPUs) para entrenar DNNs con muchos datos. En cada paso de entrenamiento, repartimos el minibatch entre las diferentes máquinas, cada una calcula su propio gradiente y, si el entrenamiento es síncrono:
 1. Cada máquina se bloquea hasta recibir el gradiente sumado en una máquina central
 2. Cada máquina actualiza su gradiente local (y modelo) sin esperar a recibir el gradiente sumado en una máquina central
 3. Cada máquina actualiza su gradiente local (y modelo) y espera a recibir el gradiente sumado en una máquina central
 4. Ninguna de las anteriores

La 1. La 2 es entrenamiento asíncrono o hogwild. En el caso de la 3, no tiene sentido actualizar el gradiente local (y modelo) y luego esperar a recibir el gradiente sumado en una máquina central.

13.5. Regularización

- ▶ La penalización de pesos (regularización ℓ_2) y la terminación temprana son técnicas usuales para evitar el sobre-entrenamiento de redes. Otra técnica usual es dropout; en relación con ella, indica la respuesta correcta.
 1. Por cada conexión de entrada de cada neurona, ésta se apaga aleatoriamente, según cierta probabilidad dada, p
 2. Por cada conexión de salida de cada neurona, ésta se apaga aleatoriamente, según cierta probabilidad dada, p
 3. Por cada neurona, ésta se apaga (todas sus conexiones de salida) aleatoriamente, según cierta probabilidad dada, p
 4. Ninguna de las anteriores

La 3.